

Módulo 03 - Exercício 07

Funções de Janela – Cláusula OVER

Tempo previsto: 90min

As respostas dos exercícios se encontram no arquivo M03E07 - Exercício Funcoes de Janela.sql.

Introdução

As funções de janela (window functions) permitem realizar cálculos sobre um conjunto de linhas relacionadas à linha atual, sem recolher o resultado em uma única linha como fazem as funções de agregação com GROUP BY. São definidas com a cláusula OVER, que especifica como as linhas são particionadas e ordenadas para o cálculo.

Sintaxe geral:

```
FUNCAO() OVER (  
    [PARTITION BY coluna]  
    [ORDER BY coluna [ASC|DESC]]  
    [ROWS|RANGE BETWEEN ... AND ...]  
)
```

Funções abordadas neste exercício:

- Classificação: RANK, DENSE_RANK, ROW_NUMBER, NTILE
- Análise: LAG, LEAD, FIRST_VALUE, LAST_VALUE
- Agregação: SUM com OVER (sem e com PARTITION BY)

Configuração Inicial

Abra o SQL Server Management Studio, faça login e abra uma janela de query selecionando New Query na barra de tarefas. Na barra de ferramentas, selecione o banco de dados AdventureWorks, ou execute o comando abaixo:

```
USE AdventureWorks
```

Exercício 1 – Funções de Classificação: RANK, DENSE_RANK e ROW_NUMBER

1. Utilize uma CTE chamada CTE_VendasProduto para calcular o total vendido (SUM de LineTotal) por subcategoria e produto, fazendo JOINS entre as tabelas Sales.SalesOrderDetail, Production.Product e Production.ProductSubcategory. Agrupe por Subcategoria e Produto.
2. Na consulta principal, aplique as três funções de classificação com PARTITION BY pelo nome da subcategoria, ordenando pelo TotalVendido de forma decrescente. Exiba as colunas: Subcategoria, Produto, TotalVendido, Rank, DenseRank e RowNumber. Ordene por Subcategoria e TotalVendido DESC.

3. Observe atentamente as diferenças entre RANK, DENSE_RANK e ROW_NUMBER quando existirem valores de TotalVendido iguais dentro de uma mesma subcategoria.

Resultado esperado:

	Subcategoria	Produto	TotalVendido	Rank	DenseRank	RowNumber
1	Bib-Shorts	Men's Bib-Shorts, M	86166.045931	1	1	1
2	Bib-Shorts	Men's Bib-Shorts, S	47214.369376	2	2	2
3	Bib-Shorts	Men's Bib-Shorts, L	34178.202000	3	3	3
4	Bike Racks	Hitch Rack - 4-Bike	237096.156000	1	1	1
5	Bike Stands	All-Purpose Bike Stand	39591.000000	1	1	1
6	Bottles and Cages	Water Bottle - 30 oz.	28654.163327	1	1	1
7	Bottles and Cages	Mountain Bottle Cage	20229.750000	2	2	2
8	Bottles and Cages	Road Bottle Cage	15390.880000	3	3	3

266 linhas

Exercício 2 – NTILE: Classificação de Clientes por Faixa de Receita

1. Escreva uma consulta com uma CTE chamada CTE_ComprasPorCliente que calcule o total comprado (SUM de TotalDue) por cliente, fazendo JOIN entre Sales.SalesOrderHeader e Person.Person. Agrupe pelo nome completo do cliente.

2. Na consulta principal, utilize NTILE(4) com ORDER BY TotalCompras para dividir os clientes em 4 grupos (quartis), do menor para o maior volume de compras. Adicione uma coluna calculada chamada FaixaCliente que converta o número do quartil em texto, conforme a tabela abaixo:

Quartil	FaixaCliente
1	Bronze
2	Prata
3	Ouro
4	Platina

3. Exiba: NomeCompleto, TotalCompras, Quartil e FaixaCliente. Ordene pelo TotalCompras de forma decrescente.

Resultado esperado:

	NomeCompleto	TotalCompras	Quartil	FaixaCliente
1	Jenny Xu	18027,8431	4	Platina
2	Jon Xu	15799,0912	4	Platina
3	Hannah Clark	14691,395	4	Platina
4	Taylor Jones	14690,1686	4	Platina
5	Hannah Lee	14662,5435	4	Platina
6	Kelli Chander	14658,9191	4	Platina

9775 linhas**Exercício 3 – LAG e LEAD: Evolução de Vendas Anuais por Vendedor**

1. Utilize uma CTE chamada CTE_VendasAnuais para calcular o total vendido (SUM de TotalDue) por vendedor e por ano (YEAR de OrderDate), fazendo JOIN entre Sales.SalesOrderHeader e Person.Person. Filtre apenas pedidos com SalesPersonID não nulo. Agrupe por NomeVendedor e AnoVenda.

2. Na consulta principal, aplique as seguintes funções de análise, todas com PARTITION BY NomeVendedor ORDER BY AnoVenda:

- LAG(TotalVendas, 1, 0) → coluna TotalAnoAnterior
- LEAD(TotalVendas, 1, 0) → coluna TotalProximoAno
- TotalVendas - LAG(...) → coluna Variacao
- (Variacao / LAG(...)) * 100 → coluna PercVariacao

Obs: utilize NULLIF no denominador de PercVariacao para evitar divisão por zero no primeiro ano de cada vendedor.

3. Exiba: NomeVendedor, AnoVenda, TotalVendas, TotalAnoAnterior, TotalProximoAno, Variacao e PercVariacao. Ordene por NomeVendedor e AnoVenda.

Resultado esperado:

	NomeVendedor	AnoVenda	TotalVendas	TotalAnoAnterior	TotalProximoAno	Variacao	PercVariacao
1	Amy Alberts	2012	130667,0249	0,00	632002,5989	130667,0249	NULL
2	Amy Alberts	2013	632002,5989	130667,0249	63747,8429	501335,574	383,67
3	Amy Alberts	2014	63747,8429	632002,5989	0,00	-568254,756	-89,91
4	David Campbell	2011	675663,694	0,00	1453583,0411	675663,694	NULL
5	David Campbell	2012	1453583,0411	675663,694	1525864,3155	777919,3471	115,13
6	David Campbell	2013	1525864,3155	1453583,0411	552783,5519	72281,2744	4,97
7	David Campbell	2014	552783,5519	1525864,3155	0,00	-973080,7636	-63,77

58 linhas**Exercício 4 – FIRST_VALUE e LAST_VALUE: Primeiro e Último Pedido por Cliente**

1. Escreva uma consulta usando a tabela Sales.SalesOrderHeader com JOIN em Person.Person para obter o nome completo do cliente (NomeCompleto).
2. Para cada linha de pedido, utilize funções de análise com PARTITION BY CustomerID ORDER BY OrderDate para exibir:
 - FIRST_VALUE(OrderDate) → coluna PrimeiroPedidoData
 - LAST_VALUE(OrderDate) → coluna UltimoPedidoData

Atenção: LAST_VALUE opera por padrão apenas até a linha atual da janela. Para obter corretamente o último valor da partição, defina o frame explicitamente com a cláusula:

```
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
```

3. Exiba: NomeCompleto, SalesOrderID, OrderDate, TotalDue, PrimeiroPedidoData e UltimoPedidoData. Ordene por NomeCompleto e OrderDate.

Resultado esperado:

	NomeCompleto	SalesOrderID	OrderDate	TotalDue	PrimeiroPedidoData	UltimoPedidoData
1	Aaron Adams	67738	2014-03-06 00:00:00.000	2632,0437	2014-03-06 00:00:00.000	2014-03-06 00:00:00.000
2	Aaron Allen	62687	2013-12-23 00:00:00.000	1918,2579	2013-12-23 00:00:00.000	2013-12-23 00:00:00.000
3	Aaron Baker	55415	2013-08-31 00:00:00.000	16,8292	2013-08-31 00:00:00.000	2013-08-31 00:00:00.000
4	Aaron Campbell	64304	2014-01-16 00:00:00.000	36,023	2014-01-16 00:00:00.000	2014-05-20 00:00:00.000
5	Aaron Campbell	73339	2014-05-20 00:00:00.000	77,3169	2014-01-16 00:00:00.000	2014-05-20 00:00:00.000

17563 linhas**Exercício 5 – SUM com OVER: Participação Percentual por Território**

1. Escreva uma consulta usando a tabela Sales.SalesOrderHeader com JOIN em Sales.SalesTerritory para obter o nome do território (Territorio).
2. Para cada pedido, utilize SUM com a cláusula OVER para calcular:

- SUM(TotalDue) OVER (PARTITION BY TerritoryID) → coluna TotalTerritorio
- SUM(TotalDue) OVER () → coluna TotalGeral (sem partição)
- TotalDue / TotalTerritorio * 100 → coluna PercNoTerritorio
- TotalTerritorio / TotalGeral * 100 → coluna PercNoTotal

Obs: observe que SUM OVER () sem PARTITION BY calcula o grande total de todos os pedidos, permitindo calcular a fatia que cada território representa no total geral.

3. Exiba: Territorio, SalesOrderID, OrderDate, TotalDue, TotalTerritorio, TotalGeral, PercNoTerritorio e PercNoTotal. Ordene por Territorio e TotalDue DESC.

Resultado esperado:

	Territorio	SalesOrderID	OrderDate	TotalDue	TotalTerritorio	TotalGeral	PercNoTerritorio	PercNoTotal
1	Australia	53566	2013-07-31 00:00:00.000	71729,8592	11814376,0952	123216786,1159	0,60	9,58
2	Australia	53485	2013-07-31 00:00:00.000	65910,6732	11814376,0952	123216786,1159	0,55	9,58
3	Australia	51814	2013-06-30 00:00:00.000	65741,4458	11814376,0952	123216786,1159	0,55	9,58
4	Australia	71805	2014-05-01 00:00:00.000	65263,2967	11814376,0952	123216786,1159	0,55	9,58
5	Australia	65191	2014-01-29 00:00:00.000	63426,4562	11814376,0952	123216786,1159	0,53	9,58
6	Australia	57075	2013-09-30 00:00:00.000	57438,3505	11814376,0952	123216786,1159	0,48	9,58
7	Australia	58931	2013-10-30 00:00:00.000	55204,4412	11814376,0952	123216786,1159	0,46	9,58
8	Australia	69550	2014-03-31 00:00:00.000	46884,3917	11814376,0952	123216786,1159	0,39	9,58
9	Australia	59018	2013-10-30 00:00:00.000	45190,4579	11814376,0952	123216786,1159	0,38	9,58

31465 linhas